


# Bases de Dados

## ÍNDICES

©José Barateiro 2024
Adaptado de Silberschatz, Korth e Sudarschan

# Hashing

# Hash estático

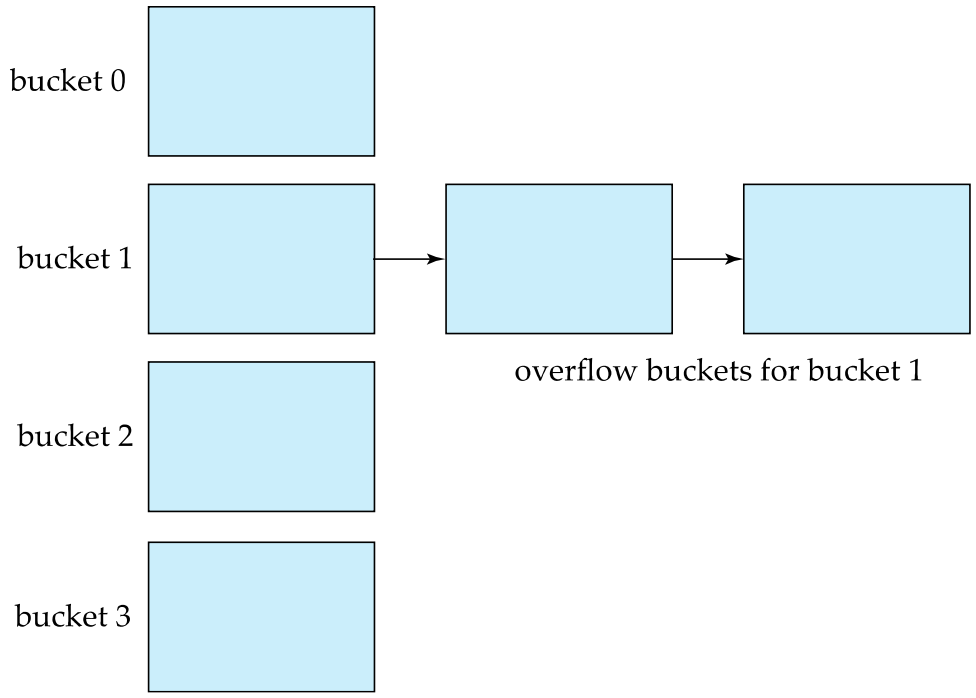
- Um **bucket** é uma unidade de armazenamento que contém uma ou mais entradas (um bucket é normalmente um bloco de disco).
  - obtemos o bucket de uma entrada a partir da *search key* usando uma **função de hash**
- A função de hash *h* é uma função do conjunto de todos os valores de search keys *K* para o conjunto de todos os endereços dos buckets *B.*
- A função hash é usada para localizar entradas para acesso, inserção e eliminação.
- Entradas com diferentes valores de *search keys* podem ser associadas ao mesmo bucket; portanto, o bucket deve ser pesquisado sequencialmente para localizar uma entrada.
- Num **índice hash**, os buckets armazenam entradas com ponteiros para registos

# Gestão de bucket overflow

- O bucket overflow pode ocorrer devido a
  - Buckets insuficientes
  - Distorção na distribuição de registos. Pode ocorrer devido a dois motivos:
    - vários registos têm o mesmo valor de search key
    - a função hash escolhida produz uma distribuição não uniforme de valores
- Embora a probabilidade de bucket overflow possa ser reduzida, não pode ser eliminada; é tratada usando ***overflow buckets***.

# Gestão de bucket overflow (cont.)

- **Overflow chaining** – os overflow de um determinado bucket são encadeados numa lista ligada. Este esquema denomina-se **closed addressing** ou **closed hashing**.



# Exemplo de organização do hash file

Organização do hash file para a relação *instrutor*, usando *dept_name* como search key.

bucket 0

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

bucket 1

| | | | |
|---|---|---|---|
| 15151 | Mozart | Music | 40000 |
| | | | |
| | | | |
| | | | |

bucket 2

| | | | |
|---|---|---|---|
| 32343 | El Said | History | 80000 |
| 58583 | Califieri | History | 60000 |
| | | | |
| | | | |

bucket 3

| | | | |
|---|---|---|---|
| 22222 | Einstein | Physics | 95000 |
| 33456 | Gold | Physics | 87000 |
| 98345 | Kim | Elec. Eng. | 80000 |
| | | | |

bucket 4

| | | | |
|---|---|---|---|
| 12121 | Wu | Finance | 90000 |
| 76543 | Singh | Finance | 80000 |
| | | | |
| | | | |

bucket 5

| | | | |
|---|---|---|---|
| 76766 | Crick | Biology | 72000 |
| | | | |
| | | | |
| | | | |

bucket 6

| | | | |
|---|---|---|---|
| 10101 | Srinivasan | Comp. Sci. | 65000 |
| 45565 | Katz | Comp. Sci. | 75000 |
| 83821 | Brandt | Comp. Sci. | 92000 |
| | | | |

bucket 7

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

# Deficiências do Static Hashing

- No static hashing, a função *h* mapeia valores de search keys para um conjunto fixo de *B* endereços de buckets. As BDs crescem ou encolhem com o tempo.
  - Se o número inicial de buckets for muito pequeno e a BD aumentar, o desempenho será prejudicado devido a muitos overflows.
  - Se for alocado espaço para o crescimento previsto, uma quantidade significativa de espaço será desperdiçada inicialmente (e os buckets ficarão subutilizados).
  - Se a BD diminuir, espaço será desperdiçado.
- Solução: reorganização periódica com uma nova função hash
  - Caro, interrompe as operações normais
- Melhor solução: permitir que o número de buckets seja modificado dinamicamente.

# Acesso de Multiple-Keys

- Usar vários índices para determinados tipos de consultas.
- Exemplo:

  **select** *ID*

  **from** *instructor*

  **where** *dept_name* = “Finance” **and** *salary* = 80000

- Estratégias possíveis para processar consultas usando índices em atributos únicos:
  1. Use o índice em *dept_name* para encontrar instrutores com o nome de departamento Finance; testar se salário = *80000*
  2. Use o índice sobre *salário* para encontrar instrutores com um salário de 80000; teste *dept_name* = “Finance” .
  3. Use o índice *dept_name* para encontrar ponteiros para todos os registos pertencentes ao departamento “Finance” . Da mesma forma, use o índice em *salary* . Intersete os conjuntos de ponteiros obtidos.

# Índices em multiple-keys

**Composite search keys** são search keys que contêm mais de um atributo

- Por exemplo (*dept_name, salary*)

Ordenação lexicográfica: $(a_1, a_2) < (b_1, b_2)$ se qualquer um

- $a_1 < b_1$, ou
- $a_1 = b_1$ e $a_2 < b_2$

# Índices em multiple-keys

Suponha que temos um índice em (*dept_name, salary*).

- Com a cláusula **where**

  **where** *dept_name* = “Finance” **and** *salary* = 80000

  o índice em (*dept_name, salary* ) pode ser usado para obter os registos que satisfaçam ambas as condições.
  - Usar índices separados é menos eficiente — podemos obter muitos registos (ou ponteiros) que satisfazem apenas uma das condições.

- Também pode lidar de forma eficiente com

  **where** *dept_name* = “Finance” **and** *salary* < 80000

# Outras características

**Covering indices**

- Adiciona atributos extra ao índice para que (algumas) consultas possam evitar o acesso aos registos reais
- Armazena atributos extras somente nas folhas
  - Porquê?

Particularmente útil para *índices non-clustered*

- Porquê?

# Criação de Índices

- Exemplo
  **create index** *takes_pk* **on** *takes* (*ID,course_ID, year, semester, section*)
  **drop index** *takes_pk*
- A maioria dos SGBD permite a especificação do tipo de índice *clustering*
- Índices na chave primária criados automaticamente por alguns SBDD
  - Porquê?
- Alguns SGBD também criam índices em atributos de chave estrangeira
- Os índices podem acelerar muito as pesquisas, mas impõem custos às atualizações
  - A escolha dos índices deve basear-se na carga de trabalho de consulta e de atualização

# Definição de índice em SQL

- Criar um índice

  **create index** <index-name> **on** <relation-name>
  (<attribute-list>)

  E.g.,: **create index** *b-index* **on** *branch(branch_name)*

- Use **create unique index** para especificar e impor indiretamente a condição de que a chave de pesquisa seja uma chave candidata.

- Para remover um índice

  **drop index** <index-name>

- A maioria dos SGBD permite a especificação do tipo de índice e se é clustered ou non-clustered.

# Bitmap Indices

- Os índices de bitmap são um tipo especial de índice projetado para consultas eficientes em multiple-keys
- Os registos numa relação consideram-se numerados sequencialmente
  - Dado um número *n,* deve ser fácil recuperar o registo *n*
    - Particularmente fácil se os registos forem de tamanho fixo
- Aplicável em atributos que assumem um número relativamente pequeno de valores distintos
  - Por exemplo, género, país, estado, …
  - Por exemplo, nível de vencimento anual (vencimento dividido num pequeno número de níveis, como 0-9999, 10000-19999, 20000-50000, 50000-infinito)
- Um bitmap é simplesmente uma matriz de bits

# Bitmap Indices (cont.)

- Na sua forma mais simples, um índice de bitmap num atributo tem um bitmap para cada valor do atributo
  - Bitmap tem tantos bits quanto registos
  - Num bitmap para valor v, o bit para um registo é 1 se o registo tiver o valor v para o atributo e é 0 caso contrário

Exemplo

| record number | *ID* | *gender* | *income_level* |
|---|---|---|---|
| 0 | 76766 | m | L1 |
| 1 | 22222 | f | L2 |
| 2 | 12121 | f | L1 |
| 3 | 15151 | m | L4 |
| 4 | 58583 | f | L3 |

Bitmaps for *gender*

| | |
|---|---|
| m | 10010 |
| f | 01101 |

Bitmaps for *income_level*

| | |
|---|---|
| L1 | 10100 |
| L2 | 01000 |
| L3 | 00001 |
| L4 | 00010 |
| L5 | 00000 |

# Índices de bitmap (cont.)

- Os índices de bitmap são úteis para consultas em vários atributos
  - não são particularmente úteis para consultas de atributos únicos
- As consultas são respondidas usando operações de bitmap
  - Intersecção (and)
  - União (or)
- Cada operação usa dois bitmaps do mesmo tamanho e aplica a operação em bits correspondentes para obter o bitmap resultante
  - Por exemplo , 100110 AND 110011 = 100010

    100110 OR 110011 = 110111
    NOT 100110 = 011001
  - Homens com nível de renda L1: 10010 AND 10100 = 10000
    - Pode então recuperar os registos necessários.
    - Contar o número de registos correspondentes é ainda mais rápido